

Module III :- III Topic III :- Agile testing and Defect Management

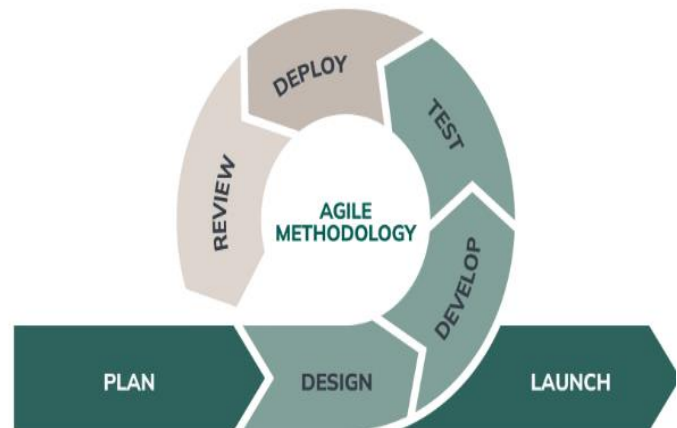
Agile Testing and Agile principles and values

Agile Testing :-Definition_

Agile Testing is a continuous testing approach in Agile development where testing is performed **throughout the software development lifecycle**, in close collaboration with developers and customers.

Key Characteristics of Agile Testing

- Early and continuous testing
- Customer collaboration
- Frequent feedback
- Adaptability to change
- Whole team involvement (developers, testers, product owner)



Agile Testing Lifecycle

1. **Test Planning** – Identify test scope for iteration/sprint
2. **Test Design** – Create test cases/user stories
3. **Test Execution** – Continuous testing during sprint
4. **Defect Reporting** – Immediate feedback
5. **Review & Retrospective** – Improve process

Agile Manifesto – Values

The **Agile Manifesto** defines **4 core values**:

1. **Individuals and interactions** over processes and tools
2. **Working software** over comprehensive documentation
3. **Customer collaboration** over contract negotiation
4. **Responding to change** over following a plan

Agile Principles:-The Agile Manifesto defines 12 principles:

1. Customer satisfaction through early and continuous delivery
2. Welcome changing requirements, even late in development
3. Deliver working software frequently
4. Business people and developers work together daily
5. Build projects around motivated individuals
6. Face-to-face conversation is the most effective communication
7. Working software is the primary measure of progress
8. Sustainable development pace
9. Continuous attention to technical excellence and good design
10. Simplicity—maximize the amount of work not done
11. Best architectures and designs emerge from self-organizing teams
12. Regular reflection and adjustment (retrospectives)

Agile Testing Quadrants

The **Agile Testing Quadrants**, proposed by **Brian Marick**, categorize testing based on whether tests are **business-facing or technology-facing** and whether they **support development or critique the product**.

Quadrant Description:

Q1 – Technology-facing & supports the team

Includes **unit tests, component tests, and API tests**. These are mostly **automated** and help developers build the right code.

Q2 – Business-facing & supports the team

Includes **functional testing, acceptance testing, and user story validation**. These tests ensure the software meets **business requirements**.

Q3 – Business-facing & critiques the product

Includes **exploratory testing, usability testing, and UAT**. These tests focus on **real user behavior and experience**.

Q4 – Technology-facing & critiques the product

Includes **performance, load, stress, and security testing**. These tests validate **non-functional requirements**.

Example: Online Shopping Application

Q1 – Technology-facing & supports the team

- Example: Writing **unit tests** to validate price calculation and discount logic using JUnit.

- Ensures the internal code works correctly.

Q2 – Business-facing & supports the team

- Example: **Functional testing** to verify that a user can add items to the cart and place an order as per requirements.
- Confirms business rules are satisfied.

Q3 – Business-facing & critiques the product

- Example: **Exploratory testing** of the checkout process to check usability and user experience.
- Identifies real-world usage issues.

Q4 – Technology-facing & critiques the product

- Example: **Performance testing** to verify the system handles 1,000 users simultaneously during a sale.
- Ensures non-functional quality.

Defect Life Cycle and

The Defect Life Cycle (Bug Life Cycle)

This is the specific set of states a defect goes through from the moment it is discovered until it is permanently closed.

Key Stages:

- **New:** When a defect is first logged and posted.
- **Assigned:** A lead or manager assigns the bug to a specific developer.
- **Open:** The developer starts analyzing the bug and working on a fix.
- **Fixed:** The developer has made the code changes and verified the fix locally.
- **Pending Retest:** The defect is sent back to the tester to verify the fix in the testing environment.
- **Retest:** The tester performs the test again to see if the bug is truly gone.
- **Verified:** The tester confirms the fix is working as expected.
- **Closed:** The final state—the bug is dead and gone.

The "Detours":

- **Rejected:** The developer feels the bug isn't actually a bug (e.g., it's a feature or "works as intended").
- **Deferred:** The bug is real but won't be fixed in this specific release (usually due to low priority).
- **Duplicate:** Someone else already reported it.

Defect Classification

Classification helps teams prioritize their workload. We usually categorize bugs by **Severity** (impact on the system) and **Priority** (urgency of the fix).

1. By Severity (Technical Impact)

This measures how much the bug "breaks" the software.

Level	Description	Example
Critical	The entire system or a major module is down; no workaround.	The app crashes every time you try to log in.
Major	A significant feature is broken, but the system still runs.	The "Add to Cart" button works, but the "Remove" button doesn't.
Minor	A bug that causes some annoyance but doesn't break functionality.	A UI alignment issue or a broken link to "Terms of Service."
Low	Cosmetic issues that don't affect usage.	A typo in a header or the wrong shade of blue on a button.

2. By Priority (Business Urgency)

This is about "When do we need to fix this?"

- **P1 (High):** Must be fixed immediately (usually linked to Critical severity).
- **P2 (Medium):** Should be fixed in the next build/iteration.
- **P3 (Low):** Fix it if we have time before the final release.

Note: A bug can be **High Severity** but **Low Priority**. For example, a crash that only happens on an ancient version of Windows that only 0.01% of users use. It's a "Critical" crash, but not a "High" priority fix.

Defect Report:-

A **Defect Report** (or Bug Report) is a formal document that tells a developer exactly what is wrong, how to find it, and how bad it is. A well-written report reduces the "back-and-forth" between teams and gets the fix deployed faster.

The Anatomy of a Perfect Defect Report

Here is a standard template used in industry tools like Jira or Azure DevOps.

1. Basic Information

- **Defect ID:** Unique identifier (e.g., BUG-101).
- **Title/Summary:** A concise "what, where, and when."
 - *Bad:* "The login is broken."
 - *Good:* "Login page throws 500 error when using special characters in password."
- **Reporter:** Your name.
- **Date:** When it was found.

2. Classification

- **Severity:** Critical / Major / Minor / Low.
- **Priority:** High / Medium / Low.
- **Environment:** OS (iOS 17), Browser (Chrome v122), Device (iPhone 15).

3. The "Meat" of the Report

- **Steps to Reproduce:** A numbered list of exactly what you clicked.
 1. Navigate to `www.example.com/login`.
 2. Enter a valid email address.
 3. Enter `Pass!word@123` in the password field.
 4. Click the "Login" button.
- **Expected Result:** "User should be redirected to the Dashboard."
- **Actual Result:** "System displays 'Internal Server Error (500)' and stays on login page."

4. Evidence & Context

- **Attachments:** Screenshots with red circles around the error, screen recordings, or log files.
- **Build Version:** The specific version of the code you were testing (e.g., `v2.4.0-rc1`).

Sample Defect Report

Field	Details
ID	BUG-402
Summary	Search results page fails to load when search query exceeds 50 characters.
Component	Search Module / Front-end
Severity	Major (Core functionality fails for specific inputs)
Priority	Medium (Uncommon user behavior, but needs fixing)
Steps	1. Go to Home Page. 2. Paste a string of 60 'A' characters into search bar. 3. Press Enter.
Expected	Results found or "No results found" message.
Actual	Page turns white (Blank Page) and console shows a Timeout error.

Defect Management:-

Defect Management is the high-level process of identifying, managing, and resolving bugs efficiently. While the "Defect Life Cycle" is the path a single bug takes, Defect Management is the **entire strategy** a team uses to keep software quality under control.

Without a solid management process, teams often fall into "Bug Debt," where they are so busy fixing old mistakes that they can't build new features.

The Defect Management Process

The process is generally broken down into these five key stages:

1. Discovery (Identification) :-The team finds as many defects as possible. These can come from:

- **Manual Testing:** Testers clicking through the app.
- **Automated Testing:** Scripts failing during a build.
- **Unit Testing:** Developers finding issues during coding.

2. Categorization & Prioritization:-Not all bugs are created equal. In this stage, the **Defect Triage Committee** (usually a Project Manager, a Test Lead, and a Dev Lead) meets to:

- Confirm the bug is valid.
- Assign **Severity** and **Priority**.
- Decide if it will be fixed now or **Deferred** for later.

3. Resolution (The Fix):-The defect is moved into the "Open" state.

- The developer reproduces the issue.
- The code is corrected.
- A "Fix" notification is sent back to the testing team.

4. Verification:-The tester doesn't just take the developer's word for it!

- **Retesting:** Running the exact same steps to ensure the bug is gone.
- **Regression Testing:** Checking *other* parts of the system to make sure the fix didn't break something else.

5. Closure & Reporting:-Once verified, the defect is closed. The data from these bugs is then used for **Defect Reporting**, where managers look at:

- **Defect Density:** How many bugs per module?
- **Defect Leakage:** How many bugs did we miss that customers found?

Test Scenario:-

While a **Defect Report** focuses on what went wrong, a **Test Scenario** focuses on what needs to be checked. Think of it as the "high-level" goal of testing.

What is a Test Scenario?

A Test Scenario is a high-level classification of testable requirements. It is essentially a one-line statement describing "**What to test**" without going into the granular "how" (which is what a Test Case does).

Analogy: > * **Test Scenario:** Check the functionality of the car's braking system.

- **Test Case:** Step 1: Accelerate to 40mph. Step 2: Slam the brakes. Step 3: Measure if the car stops within 20 feet.

The Hierarchy of Testing :- To understand where scenarios fit, look at this structure:

1. **Requirement:** User must be able to log in.
2. **Test Scenario:** Verify the Login functionality with different credentials.
3. **Test Case 1:** Login with valid username and password.
4. **Test Case 2:** Login with invalid password.
5. **Test Case 3:** Login with an empty username.

Why use Test Scenarios?

- **Better Coverage:** It's easier to list 50 scenarios to ensure you haven't missed a feature than to write 500 detailed test cases first.
- **Time Management:** If time is short, testers can perform "Scenario-based testing" to cover the most important paths.
- **Simplicity:** Stakeholders (like Product Owners) can read scenarios to quickly understand what is being tested without getting bogged down in technical steps

Real-World Examples (E-Commerce App)

Module Test Scenario

Search Verify that search results are relevant to the keywords entered.

Search Verify the behavior of "Auto-suggest" while typing.

Payments Verify that the user can pay using a Credit Card.

Payments Verify the system behavior when a transaction is declined by the bank.

Orders Verify that the user receives an email confirmation after a successful order.

How to write a good Test Scenario:-

1. **Read the Requirements:** Understand the Business Requirement Specification (BRS) or Functional Requirement Specification (FRS).
2. **Identify User Flows:** Think like an end-user. What are the common paths they take? What are the "edge cases"?
3. **Keep it Simple:** One scenario should focus on one specific functional area.
4. **Traceability:** Ensure every scenario maps back to a requirement so you can prove the feature was tested.

Test Scenario Template & Examples :-

Template

ID	Scenario	ID	Requirement	Scenario Description
	TS_01		REQ_Login_01	Verify login with various valid/invalid inputs.

Design Test case for given test AND design test case in excel:-

Column Header	Description	
Test Case ID	Unique identifier (e.g., TC_LOG_01).	
Test Priority	Low, Medium, High, or Critical.	
Test Title	A short summary of what is being tested.	
Pre-conditions	Requirements before starting (e.g., "User is on the home page").	
Test Steps	Step-by-step instructions for the tester.	
Test Data	Specific inputs used (e.g., username, password).	
Expected Result	What should happen if the app works correctly.	
Actual Result	What actually happened (filled during execution).	
Status	Pass, Fail, Blocked, or N/A.	

2. Example: Login Page Test Cases

If your application has a login feature, here is how you would design those cases in your spreadsheet:

TC ID	Priority	Test Title	Test Steps	Expected Result	
TC_01	High	Verify Login with Valid Credentials	1. Enter valid email.2. Enter valid password.3. Click "Login".	User is redirected to the Dashboard successfully.	
TC_02	High	Verify Login with Invalid Password	1. Enter valid email.2. Enter wrong password.3. Click "Login".	Error message "Invalid credentials" appears; user stays on login page.	
TC_03	Medium	Verify Password Masking	1. Type characters in the Password field.	Characters should be displayed as dots or asterisks for security.	